Dividir e Conquistar

Quicksort

Quicksort

Dividir e Conquistar

Dividir

Lista L[m...p] é dividida em L[m...j-1] e L[j+1...p] tais que os elementos de L[m...j-1] são menores que o elemento L[j], os elementos de L[j+1...p] são maiores que o elemento L[j]. L[j] é definido no procedimento Pivô(L), mas tem sua posição j corretamente estabelecida como resultado de Particiona(L,m,p).

Obs. L[j] é chamado de pivô.

Quicksort

Dividir e Conquistar

Conquistar

Listas $L[m...j-1]$ e $L[j+1...p]$ são ordenadas por chamadas recursivas a *quicksort*.

Combinar

Não precisa trabalho para combinar. A lista está ordenada.

Quicksort

```
Quicksort(L,m,p)
   se m >= p então retorna (L)
   k ← pivô(L)
   se k ≠ 0 então
     Troca(L[m],L[k])
     j ← Particiona(L,m,p)
     L1 ← Quicksort(L,m,j-1)
     L2 ← Quicksort(L,j+1,p)
  retorna(L)
```

Quicksort

pivô(*L*) – retorna o índice do maior número entre os dois primeiros diferentes em *L*

Se todos os elementos de *L* forem iguais, retorna 0 indicando que a lista já está ordenada.

Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$

Repita

  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$

  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$

  se $i < j$ então Troca($L[i]$,$L[j]$)

Até $i > j$

$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$

retorna(j,L)

Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
    enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
    enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
    se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

L: lista
m: índice do início
p: índice do fim

101	20	350	140	5	75	82	9	100	120

Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
    enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
    enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
    se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

L: lista
m: = 1 índice do início
p = 10: índice do fim

v = 101
i = 2
j = 10

101	20	350	140	5	75	82	9	100	120

Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 2$
$j = 10$



Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 3$
$j = 10$



Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 3$
$j = 10$



Quicksort

```
Particiona(L,m,p)
  v ← L[m]; i ← m+1; j ← p
  Repita
     enquanto (i ≤ p e L[i] < v) faça i ← i + 1
     enquanto (j ≥ m e L[j] ≥ v) faça j ← j - 1
     se i < j então Troca(L[i],L[j])
   Até i > j
  L[m] ← L[j] ; L[j] ←v
  retorna(j,L)
```

$v = 101$

$i = 3$

$j = 9$



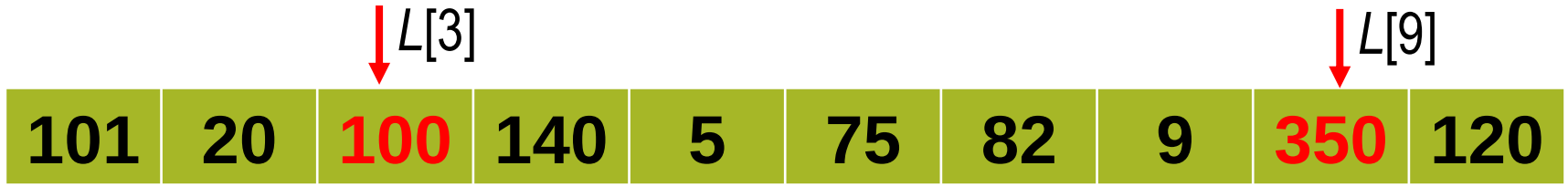
Quicksort

```
Particiona(L,m,p)
  v ← L[m]; i ← m+1; j ← p
  Repita
      enquanto (i ≤ p e L[i] < v) faça i ← i + 1
      enquanto (j ≥ m e L[j] ≥ v) faça j ← j - 1
      se i < j então Troca(L[i],L[j])
   Até i > j
  L[m] ← L[j] ; L[j] ←v
  retorna(j,L)
```

$v = 101$

$i = 3$

$j = 9$



Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$

Repita

enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$

enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$

se $i < j$ então Troca($L[i]$,$L[j]$)

Até $i > j$

$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$

retorna(*j*,*L*)

$v = 101$

$i = 3$

$j = 9$

$L[3]$

101	20	100	140	5	75	82	9	350	120

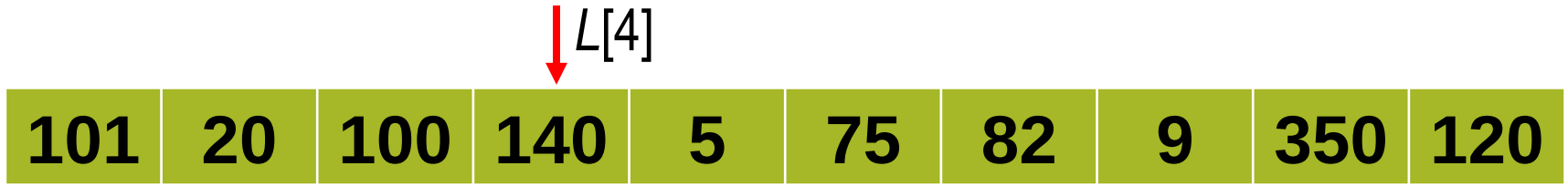
$i = 3$ 4

Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 4$
$j = 9$



Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 4$
$j = 9$

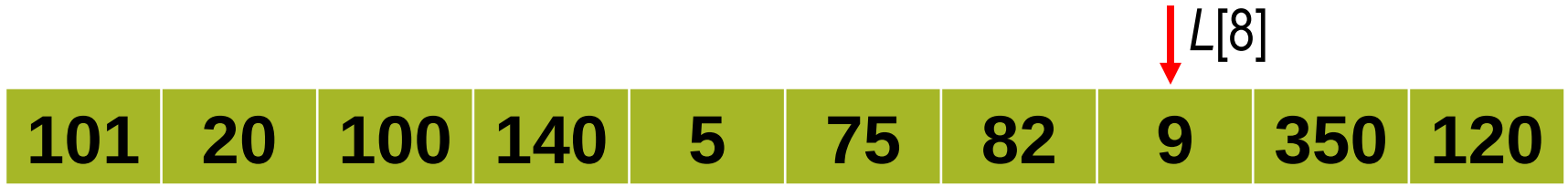


Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 4$
$j = 8$

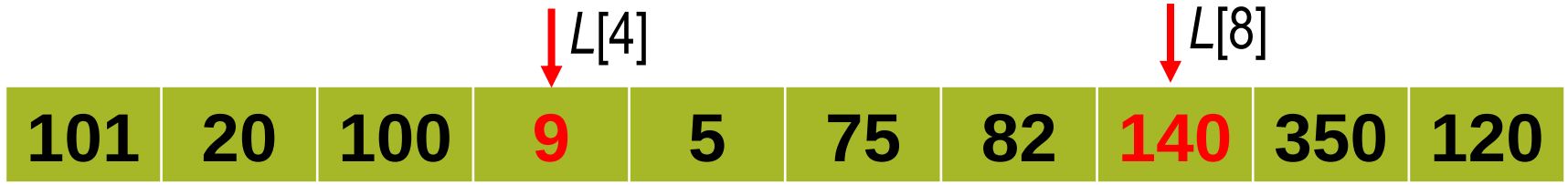


Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
    enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
    enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
    se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 4$
$j = 8$



Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$

Repita

enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$

enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$

se $i < j$ então Troca($L[i]$,$L[j]$)

Até $i > j$

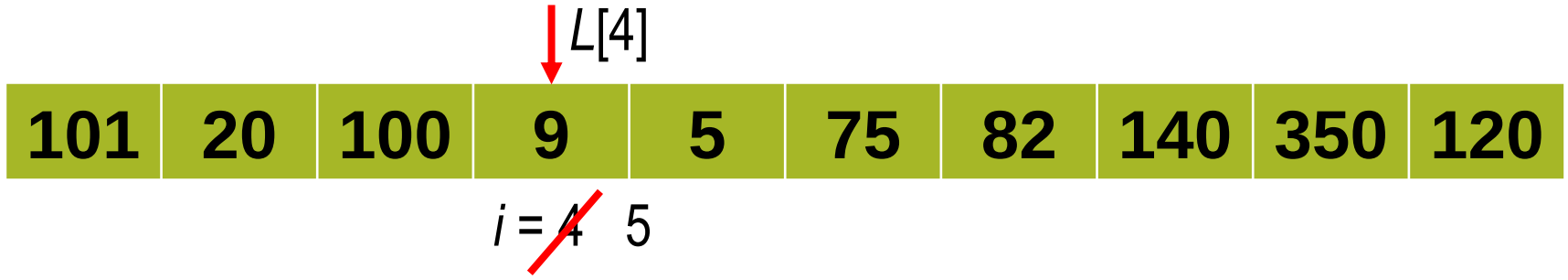
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$

retorna(*j*,*L*)

$v = 101$

$i = 4$

$j = 8$

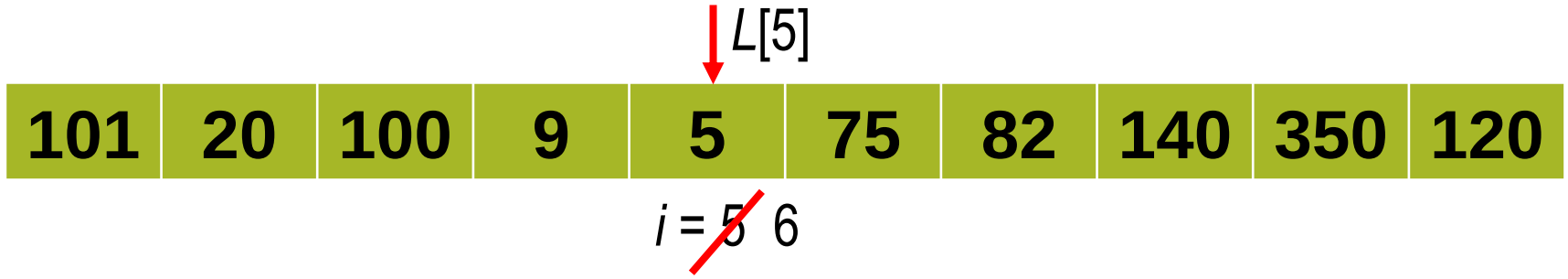


Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 5$
$j = 8$

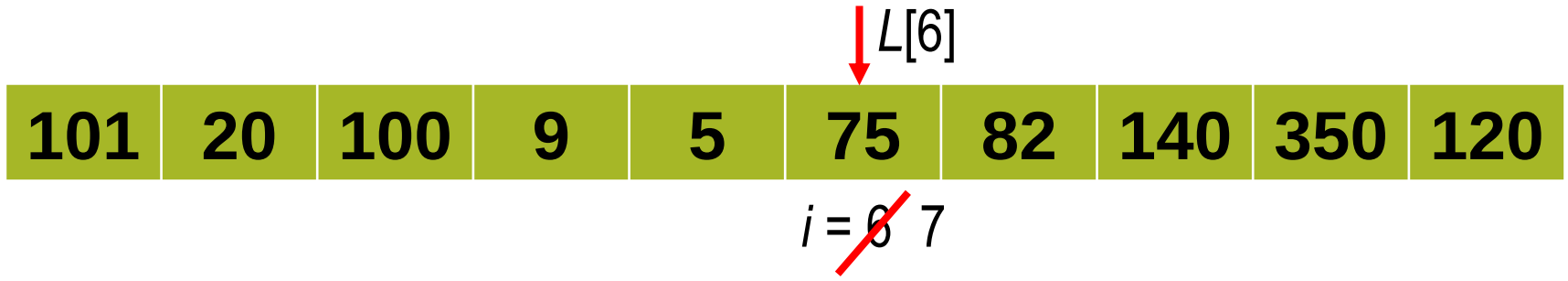


Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 6$
$j = 8$



Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$

Repita

enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$

enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$

se $i < j$ então Troca($L[i]$,$L[j]$)

Até $i > j$

$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$

retorna(*j*,*L*)

$v = 101$

$i = 7$

$j = 8$



Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 7$
$j = 8$



Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
  enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
  enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
  se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

$v = 101$
$i = 7$
$j = 7$

$i = 7$
$j = 7$

Sai do laço

$L[7]$

101	20	100	9	5	75	82	140	350	120

Quicksort

```
Particiona(L,m,p)
  v ← L[m]; i ← m+1; j ← p
  Repita
      enquanto (i ≤ p e L[i] < v) faça i ← i + 1
      enquanto (j ≥ m e L[j] ≥ v) faça j ← j - 1
      se i < j então Troca(L[i],L[j])
   Até i > j
  L[m] ← L[j] ; L[j] ←v
  retorna(j,L)
```

$v = 101$
$i = 7$
$j = 7$

FIM

82	20	100	9	5	75	101	140	350	120

Quicksort

Particiona(*L*,*m*,*p*)

$v \leftarrow L[m]$; $i \leftarrow m+1$; $j \leftarrow p$
Repita
    enquanto (i ≤ *p* e $L[i] < v$) faça $i \leftarrow i + 1$
    enquanto ($j \geq m$ e $L[j] \geq v$) faça $j \leftarrow j - 1$
    se $i < j$ então Troca($L[i]$,$L[j]$)
Até $i > j$
$L[m] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

Já está na posição final

82	20	100	9	5	75	101	140	350	120

Quicksort

Qual a complexidade de pior caso de Particiona?

Quicksort

Qual a complexidade de Particiona?

$\Theta(n)$

Exercício. Provar que complexidade de Particiona é $\Theta(n)$

Quicksort

Pior Caso

Ocorre quando após o Particiona, a lista é dividida em 1 lista com n - 1 elementos e outra com 0 elementos, e isto ocorre em todas as chamadas.

$T(n) = T(n\text{-}1) + T(0) + \Theta(n)$
$T(n) = T(n\text{-}1) + \Theta(n)$

$T(n)$ é $\Theta(n^2)$ (Veremos adiante)

Quicksort

Melhor Caso

Ocorre quando após o Particiona, a lista é dividida em 2 listas com $n/2$ elementos cada, e isto ocorre em todas as chamadas.

$T(n) \leq 2T(n/2) + \Theta(n)$

$T(n)$ é $O(n\log n)$ (Veremos adiante)

Quicksort

```
Quicksort(L,m,p)
  se m >= p então retorna (L)
  k ← pivô(L)
  se k ≠ 0 então
     Troca(L[m],L[k])
     j ← Particiona(L,m,n)
     L1 ← Quicksort(L,m,j-1)
     L2 ← Quicksort(L,j+1,p)
  retorna(L)
```

Qual a complexidade média de Quicksort, CM_Q?

Quicksort

Supor que *L* tem *n* elementos.

Temos:

- Escolha do pivô e particionamento
- Chamada do quicksort para a lista esquerda
- Chamada do quicksort para a lista direita

O desempenho depende de quem é o pivô.

Quicksort

Supor que $p(i)$ é a probabilidade do pivô ser o i+1-ésimo elemento da lista ordenada.

Então:

$$CM_Q(n) = \sum_{i=1}^{n-1} p(i)\left[CM_{Part}(n) + CM_Q(i) + CM_Q(n-i)\right]$$

O pivô será do 2º ao n-ésimo elemento

Complexidade média do particionamento

Quicksort

CM_{Part} é $O(n)$ ⟵ (Exercício. Provar)

Quicksort

CM_{Part} é $O(n)$

Temos: $$CM_Q(n) = \sum_{i=1}^{n-1} p(i)\left[n + CM_Q(i) + CM_Q(n-i)\right]$$

Quicksort

CM_{Part} é $O(n)$

Temos: $CM_Q(n) = \sum_{i=1}^{n-1} p(i)\left[n + CM_Q(i) + CM_Q(n-i)\right]$

Chamando CM_Q de T(n), temos:

$$T(n) = O\left(\sum_{i=1}^{n-1} p(i)\left[n + T(i) + T(n-i)\right]\right)$$

Quicksort

Supor todos os elementos diferentes, então o pivô será um dos dois primeiros elementos da lista, o maior deles.

Para calcular a probabilidade de que o (i+1)-ésimo elemento na ordenação esteja na primeira ou segunda posição da lista, devemos ter em mente que i elementos são menores que ele.

Quicksort

A probabilidade do (*i*+1)-ésimo elemento (ordenado) (1º, 2º, ...,nº) estar na primeira posição da lista é 1/*n* e a probabilidade do 2º ser menor

que ele é $\frac{i}{n-1}$

Quicksort

A probabilidade de o (i+1)-ésimo elemento ser o maior e estar na primeira posição é:

$$\frac{1}{n}\frac{i}{n-1}$$

Que é também a probabilidade do (i+1)-ésimo elemento estar na segunda posição e ser o maior.

Quicksort

Assim, a probabilidade de o (*i*+1)-ésimo elemento ser o pivô é:

$$\frac{2i}{n(n-1)}$$

Para $n > 1$, temos:

$$T(n)\ \acute{e}\ O\left(\sum_{i=1}^{n-1}\frac{2i}{n(n-1)}\left[n+T(i)+T(n-i)\right]\right)$$

Quicksort

Ou

$$T(n)\ \acute{e}\ O\left(\frac{2}{n(n-1)}\sum_{i=1}^{n-1} i\left[n+T(i)+T(n-i)\right]\right)$$

Quicksort

$$T(n)\ \acute{e}\ O\left(\frac{2}{n(n-1)}\sum_{i=1}^{n-1} i\left[n+T(i)+T(n-i)\right]\right)$$

Sabe-se que

$$\sum_{i=1}^{n-1} f(i)=\sum_{i=1}^{n-1} f(n-i)$$

$$\sum_{i=1}^{n-1} f(i)=\frac{1}{2}\sum_{i=1}^{n-1} f(i)+\ f(n-i)$$

Quicksort

Tomando

$$f(i) = \frac{2i}{n(n-1)}\left[n + T(i) + T(n-i)\right]$$

Temos

$$f(n-i) = \frac{2(n-i)}{n(n-1)}\left[n + T(n-i) + T(i)\right]$$

Quicksort

$$\sum_{i=1}^{n-1} f(i) = \frac{1}{2}\left[\begin{array}{l}\sum_{i=1}^{n-1} \frac{2i}{n(n-1)}\left[n + T(i) + T(n-i)\right] + \\ \sum_{i=1}^{n-1} \frac{2(n-i)}{n(n-1)}\left[n + T(n-i) + T(i)\right]\end{array}\right]$$

$$\sum_{i=1}^{n-1} f(i) = \frac{1}{(n-1)} \sum_{i=1}^{n-1}\left[n + T(i) + T(n-i)\right]$$

Quicksort

$$\sum_{i=1}^{n-1} f(i) = \frac{1}{(n-1)} \sum_{i=1}^{n-1} [n + 2T(i)]$$

$$\sum_{i=1}^{n-1} f(i) = n + \frac{2}{n-1} \sum_{i=1}^{n-1} [T(i)]$$

$$T(n) \acute{e}\ O(n) + O\left(\frac{2}{n-1} \sum_{i=1}^{n-1} [T(i)] \right)$$

Quicksort

Pode-se mostrar por indução em $n > 1$ que

$$\frac{2}{n-1}\sum_{i=1}^{n-1}[T(i)] \quad \acute{e} \quad O(n \log n)$$

Exercício. Mostrar

Portanto CM_Q é $O(n\log n)$.

FIM